

Ejercicio 23: Intercalado de Cadenas (Programación Dinámica)

Enunciado

Dadas tres cadenas de caracteres c_1 , c_2 y c_3 , queremos ver si es posible obtener c_3 intercalando los caracteres de las otras dos (sin cambiar el orden relativo).

El intercalado de dos cadenas c_1 y c_2 es una configuración donde ambas se dividen en n y m subcadenas, tales que:

$$c_1 = c1_1 + c1_2 + \dots + c1_n$$

$$c_2 = c2_1 + c2_2 + \dots + c2_m$$

$$|n - m| \leq 1$$

El intercalado es $c1_1 + c2_1 + c1_2 + c2_2 + \dots$, o bien $c2_1 + c1_1 + c2_2 + c1_2 + \dots$, donde $+$ representa la operación de concatenación.

1. Definición del Estado

Sean $n = |c_1|$ y $m = |c_2|$. Definimos la función óptima de estado como:

- $DP[i][j]$: Valor booleano (Verdadero/Falso) que indica si el prefijo de c_3 de longitud $i + j$ puede formarse mediante un intercalado válido de los primeros i caracteres de c_1 y los primeros j caracteres de c_2 .

2. Ecuación de Recurrencia

Para determinar si el estado actual $DP[i][j]$ es alcanzable, analizamos el último carácter de la secuencia de c_3 , que se encuentra en la posición $i + j$ (o índice $i + j - 1$ usando indexación de 0 a $N - 1$). Este carácter debió provenir de una de las dos cadenas:

1. **Viene de c_1 :** El carácter actual de c_1 ($c_1[i - 1]$) coincide con el de c_3 ($c_3[i + j - 1]$) y el subproblema previo descartando dicho carácter de c_1 era válido ($DP[i - 1][j] == \text{Verdadero}$).
2. **Viene de c_2 :** El carácter actual de c_2 ($c_2[j - 1]$) coincide con el de c_3 ($c_3[i + j - 1]$) y el subproblema previo descartando dicho carácter de c_2 era válido ($DP[i][j - 1] == \text{Verdadero}$).

Recurrencia General

$$DP[i][j] = (DP[i - 1][j] \wedge c_1[i - 1] == c_3[i + j - 1]) \vee (DP[i][j - 1] \wedge c_2[j - 1] == c_3[i + j - 1])$$

Casos Base e Inicialización

- **Ambas vacías:** $DP[0][0] = \text{Verdadero}$ (la cadena vacía se obtiene intercalando dos vacías).
- **Fila base (solo usar c_2):** $DP[0][j] = DP[0][j-1] \wedge (c_2[j-1] == c_3[j-1])$
- **Columna base (solo usar c_1):** $DP[i][0] = DP[i-1][0] \wedge (c_1[i-1] == c_3[i-1])$

3. Algoritmo Eficiente Iterativo (Bottom-Up)

Algoritmo EsIntercalado(c_1, c_2, c_3)

```
n ← longitud(c1)
m ← longitud(c2)

// Comprobación necesaria
Si n + m ≠ longitud(c3) entonces
    devolver FALSO
FinSi

// Crear matriz DP de tamaño (n+1) × (m+1)
Crear dp[0..n][0..m]

Para i ← 0 hasta n hacer
    Para j ← 0 hasta m hacer
        dp[i][j] ← FALSO
    FinPara
FinPara

// Caso base
dp[0][0] ← VERDADERO

// Rellenar la tabla
Para i ← 0 hasta n hacer
    Para j ← 0 hasta m hacer
        k ← i + j
        // Tomar carácter desde c1
        Si i > 0 Y
            c1[i-1] = c3[k-1] Y
            dp[i-1][j] = VERDADERO entonces
                dp[i][j] ← VERDADERO
        FinSi

        // Tomar carácter desde c2
        Si j > 0 Y
            c2[j-1] = c3[k-1] Y
            dp[i][j-1] = VERDADERO entonces
                dp[i][j] ← VERDADERO
        FinSi
    FinPara
FinPara
devolver dp[n][m]
FinAlgoritmo
```

4. Análisis de Eficiencia

- **Sin Programación Dinámica (Fuerza Bruta):** En cada posición tenemos una ramificación binaria (elegir el carácter de c_1 o de c_2). Esto da lugar a un árbol de decisión de coste exponencial:

$$\mathcal{O}(2^{n+m})$$

- **Con Programación Dinámica:**
 - **Temporal:** $\mathcal{O}(n \cdot m)$. El espacio de estados está acotado por las dimensiones de la matriz $(n + 1) \times (m + 1)$. Cada celda se calcula realizando comparaciones de tiempo constante $\mathcal{O}(1)$.
 - **Espacial:** $\mathcal{O}(n \cdot m)$ para almacenar la tabla `dp`. (Nota: Podría optimizarse a $\mathcal{O}(\min(n, m))$ si solo guardamos la fila anterior y la actual, pero la matriz completa es la respuesta óptima explicativa en un examen).